

Do the Marks Survive the Pipeline? An Empirical Study of Provenance Loss and Gain in Software Releases

Ashish, Stany, Upendra and Karthik | Target: FSE 2027

Issued 13 September 2026

Core Argument. A shipped release carries claims about its own origin in several places at once, and each is produced by a different mechanism and destroyed by a different operation. Commit trailers name the agent that authored a change, signatures bind a commit to an identity, attestations and bills of materials bind a package to its build, cards describe models and datasets, and code watermarks are proposed to mark generated source. Existing work evaluates these carriers one at a time, against an adversary, on a snippet. This paper argues that provenance is not compositional: marking every artifact does not mark the release, because the ordinary operations that assemble a release destroy origin claims unevenly and silently, with no attacker involved and no one doing anything wrong. A squash merge folds away the trailers of a branch, a rebase invalidates signatures, a rebuild from source breaks attestation linkage, and a formatter rewrites the tokens a watermark was embedded in. The same operations also manufacture claims that are false, since a human-written module passed to an assistant for a small edit returns carrying an agent trailer. If that holds, two consequences follow. Disclosure obligations that assume artifact-level marking are unmeetable as specified, because the obligation binds at the release while the evidence is destroyed before the release exists. And the empirical software engineering literature that now measures AI adoption from trailer-visible signals is resting on an instrument whose loss and gain behaviour has never been characterised.

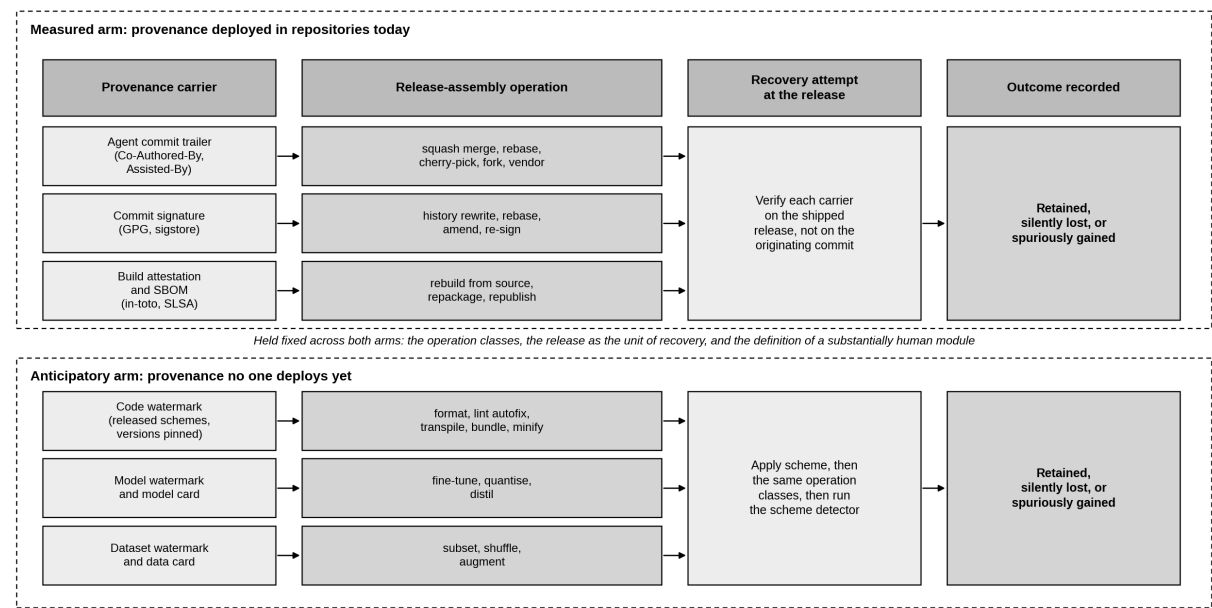


Figure 1. The two arms of the study. Provenance carrier names what holds the origin claim, release-assembly operation names the class of ordinary operation applied to it, recovery attempt states where the claim is looked for, and outcome recorded is the three-way result per carrier and operation. The measured arm covers carriers deployed in repositories today and runs on real history. The anticipatory arm covers carriers no one deploys and runs on constructed releases. Held fixed across both arms are the operation classes, the release as the unit of recovery, and the definition of a substantially human module.

Contributions

1. A characterisation of the provenance carriers a release holds, and the operation classes that destroy each one. *(the frame)*

It converts a set of unrelated mechanisms studied in separate communities into one object with one failure question, and it is what makes the release rather than the snippet the unit of analysis.

2. A measurement, on real repository history, of how often deployed origin claims survive to the release that ships them. *(the evidence base)*

This arm depends on nothing hypothetical. Trailers, signatures and attestations are emitted today and are being destroyed today, so the central result does not rest on schemes no one has adopted.

3. A measurement of spurious gain, being the rate at which origin claims attribute to a model work that is substantially human. *(the second failure direction)*

The adversarial literature has no counterpart to this direction because an attacker has no reason to add a mark. It is also the direction with the more serious consequence under any regime that treats AI generation as a category with effects.

4. An assessment of what the loss and gain rates imply for trailer-based measurement of AI adoption in empirical software engineering. *(the claim that reaches the community)*

A growing set of mining studies classifies commits by trailer. If the carrier is destroyed by squash workflows and manufactured by editor defaults, those classifications carry an unquantified error, and this paper supplies the quantity.

5. The measurement setup and the operation catalogue, released so that any future carrier or scheme can be assessed for release-level survival without rebuilding the instrument. *(the reusable artifact)*

The catalogue is derived from repository configurations rather than chosen by us, which is the property that makes it reusable and also what makes the study defensible.

The Six Questions

What is the problem?

Whether a claim about where software came from is still recoverable at the point where anyone would ask. Provenance carriers are attached to individual artifacts at the moment those artifacts are produced, and the question of origin is asked of a release, assembled later from many artifacts by operations that rewrite them. Nobody has measured whether the claims survive that assembly. The problem is not that an attacker can remove a mark, which is studied, but that sanctioned operations remove and manufacture marks with no attacker present, which is not. A recent systematic review of watermarking and attribution for AI-generated code reports that current techniques suit clean environments and that the field has favoured accuracy and invisibility over long-term resilience, which names the gap from the scheme side. A longitudinal mining study published in September 2026 names it from the measurement side, stating that its own signal cannot be disentangled from merge and squash workflows.

Why is it important?

Two constituencies depend on the answer and neither has it. The first is regulatory. Disclosure duties for AI-generated content are written as though a marked artifact stays marked, and the obligation binds at the point of release rather than at the point of generation. If the evidence is destroyed before the release exists, the duty is unmeetable as written and no amount of adversarial hardening changes that. The second is empirical software engineering itself. Mining studies now classify commits and pull requests by the presence of an agent trailer, and draw conclusions about adoption, quality and merge behaviour from the classification. One study of 33,596 pull requests across 2,807 repositories reports that agents differ enormously in whether they emit the trailer at all, with one agent using it in 1.2 percent of its pull requests while two others rely on it in over 89 and 95 percent. Another reports 62 percent of substantive edits carrying a trailer while stating that trailer-generation practice and squash workflows cannot be separated from actual involvement. The failure is two-directional. Claims are lost, so machine-written work reads as human. Claims are gained, so human work is credited to a model, and editor defaults that insert co-authorship lines when assistance was not used make this concrete rather than hypothetical.

What are the existing solutions, and why are they not enough?

Adversarial code watermark robustness. Suresh and colleagues gave the first study of robustness for watermarked LLM-generated Python, showing that semantics-preserving transformations such as variable renaming and dead code insertion erase watermarks without changing behaviour, evaluated by traversing the abstract syntax tree and applying randomised transformations. The strength we use is the measurement pattern, being apply a transformation and re-run the detector. The weakness we address is the threat model and the unit, since the transformations are chosen by the evaluator to model an attacker and the object is a snippet. It is also a two-page Tiny Papers contribution rather than a full study, so the space it closes is narrower than its citation count suggests. Syntax-aware and transformation-based schemes. STONE embeds only in non-syntax tokens to preserve keywords and structure, and SrcMarker, ACW, CodeMark, CodeIP, CLASP and more recently PromptMark embed through semantics-preserving transformation. The strength is that several are released and usable as given. The weakness shared by all of them is that each is evaluated against transformations rather than against an assembly process, and each marks one artifact in isolation. Trailer-based detection of agent involvement. A validated multi-method census across 180 million repositories separates agent-authored from agent-trailered commits and reports per-cell precision with confidence intervals and a relative-recall analysis of undercount. This is the closest work and the one we must position against rather than repeat. Its strength is the validated labelling procedure, which we adopt rather than reinvent. Its weakness for our purposes is that it characterises the signal as emitted, at the commit, and says nothing about what happens to the signal afterwards. Supply chain provenance frameworks. In-toto and SLSA define attestation formats and a step-wise model of a supply chain, and platform support has moved from opt-in towards default for public repositories. The strength is a formal vocabulary for provenance over a sequence of steps. The weakness is that the framework specifies what should be recorded and verified, not what survives when the steps are performed by ordinary teams, and its breakage behaviour under rebuild and republish is not empirically characterised.

What is the proposed idea?

An empirical study of whether origin claims survive release assembly, run over every carrier a release holds, in two arms. Input, for the measured arm. Public repositories with real history, releases with published artifacts, and the provenance carriers already present in them, being agent and co-authorship trailers, commit signatures, and build attestations and bills of materials. Input, for the anticipatory arm. Released code watermarking schemes with versions pinned, and where available model and dataset marking schemes, applied to constructed releases. Process. Derive the catalogue of release-assembly operations from repository and workflow configurations rather than inventing it, covering the history layer of squash, rebase, cherry-pick, fork and vendoring, the source layer of formatting, lint autofix, transpilation, bundling and minification, the packaging layer of rebuild, repackage and republish, and for the anticipatory arm the model and data layers. For each carrier and each operation class, record whether the claim is retained, silently lost, or spuriously gained, measured at the release rather than at the commit that produced it. Then compose operations in the orders real projects apply them, since single-operation results may not compose. Then measure spurious gain directly, by taking modules that are substantially human under a rule fixed in advance, passing them to an assistant for the kinds of small task teams delegate, and recording how often the resulting commit and release carry a claim attributing the work to a model. Output. A retention table over carriers and operation classes, a composed result, a spurious gain rate, and a statement of what the two rates imply for trailer-based measurement. What is different. The unit is a release rather than a snippet, the operations are observed rather than chosen, no adversary is assumed, and the carriers are treated together rather than one at a time. Holding the problem fixed and varying the unit of analysis is the increment here, and it is the variation that turns a security question into a software engineering one.

Why should we believe it?

The measured arm runs on provenance that exists independently of us. We did not place the trailers, signatures or attestations, and we do not control the operations that were applied to them, which removes the main freedom that would let a result be steered. The operation catalogue is derived from configurations rather than selected, and the derivation procedure, including how many repositories were sampled and how operations were ranked by prevalence, is reported as part of the result. Schemes in the anticipatory arm are used as released with versions pinned, so outcomes are attributable to the schemes rather than to a reimplementation. Labelling of agent involvement follows the validated procedure from the existing census rather than a new scheme of our own, and where we depart from it the departure is stated. Detection in the anticipatory arm is statistical, so repeated measurement and reported variance are necessary rather than optional, and single-run numbers would not support the claims. Assumptions to state. That the sampled repositories represent release practice, which requires the sampling frame to be argued rather than assumed. That detection thresholds are used as scheme authors specify, since moving a threshold moves every number. That generation settings are held fixed in the anticipatory arm, because temperature and sampling affect mark strength independently of any operation. And that a claim recorded as lost was genuinely present before the operation, which requires the before state to be established rather than inferred. The main threat to the spurious gain half is the definition of a substantially human module. It needs a stated rule, such as a proportion of unchanged lines, fixed before the experiment rather than after seeing results, and a sensitivity analysis across plausible values of the threshold.

Where are the artifacts?

The operation catalogue with its derivation from repository and workflow configurations, the sampling frame and the repository list, the measurement setup that applies operations and attempts recovery per carrier, pinned scheme, tool and model versions, the full retention table, the composed-order results, and the spurious gain materials including the substantially-human rule and its sensitivity analysis. The measurement setup and the catalogue are the reusable assets, since any future carrier or scheme can be assessed against release-level survival without rebuilding either. Under double blind the artifact is deposited in an anonymised archive with a stable identifier, with the repository list and configurations included so the sampling can be checked independently of us. A README follows the lab checklist, and the release-assembly operations are scripted so that the results regenerate rather than being reported from a one-off run.

What This Stands On

Field	Detail
Base paper	Suresh, T., Ugare, S., Singh, G., and Misailovic, S. Is Watermarking LLM-Generated Code Robust? Second Tiny Papers Track at ICLR 2024, later expanded as arXiv:2403.17983. The sentence we stand on is that semantics-preserving transformations erase watermarks from generated code without altering functionality. Relationship: port. We take their measurement pattern, apply a transformation and re-run the detector, to a setting they did not study. The transformations there are chosen by the evaluator to model an attacker and applied to a snippet. Here the operations are observed in repository configurations, applied by teams with no intent to remove anything, and the unit of recovery is a shipped release carrying several carriers at once. The setting differs enough that the result is not predictable from theirs, and a failure direction exists here, being spurious gain, that has no counterpart under an adversarial model. Note for the students: this is a two-page Tiny Paper, so its claim is narrower than the way it is usually cited, and the brief should not overstate what it settled.
Theoretical basis	Partial, and the brief states this rather than manufacturing one. The candidate frame is the in-toto model of a supply chain as a sequence of steps with attestations bound to subjects, which gives a formal vocabulary for provenance that must be preserved under a composition of steps, and for the conditions under which composition fails. The frame is presently a vocabulary rather than a result, since it does not yet yield a prediction about which operation classes destroy which carriers. It is worth one attempt before writing. The legal instrument, being the disclosure obligations for AI-generated content, supplies motivation rather than theory and should be cited as a conditional rather than asserted as settled, since whether source code falls within the marking obligation is contested. If neither the in-toto frame nor the legal instrument yields a prediction, the paper is presented as empirical without a theoretical frame, which is acceptable for a study of this kind and better than a frame fitted afterwards.
Empirical standards	Repository Mining (primary), Engineering Research, Benchmarking and Simulation, Data and artifact standard, Sampling supplement

Mapping to the SIGSOFT Empirical Standards

Standard	Why it applies	Where the work currently falls short
Repository Mining	The measured arm draws its evidence from public repository history, release artifacts and workflow configurations, and the operation catalogue is derived by mining configurations rather than by selection.	The sampling frame is undefined. The standard asks for the population, the sampling strategy and the exclusion criteria to be stated, and at present the brief has none of the three. It also asks that the mining procedure distinguish absence of a signal from absence of evidence, which is exactly the retained versus silently lost distinction, and no procedure for establishing the before state has been specified.
Engineering Research	The measurement setup is a constructed instrument, and the operation catalogue is a designed artifact intended for reuse beyond this paper.	The standard asks that the artifact be evaluated against a stated need rather than only demonstrated. No evaluation of the setup itself is designed, so there is currently no evidence that the recovery attempt per carrier is correct rather than merely implemented.
Benchmarking and Simulation	The anticipatory arm runs released schemes over constructed releases under controlled settings, which is measurement by execution rather than by observation.	The standard asks for justified selection of the systems under test and for held-fixed configuration to be reported. Scheme selection is not yet argued, versions are not pinned, and the number of repeated runs needed for stable detection is unknown until a pilot is run.
Data and artifact standard	The catalogue, the measurement setup and the retention table are the durable outputs, and the paper claims them as reusable.	Nothing is packaged yet, no anonymised deposit route is chosen, and the regeneration path from raw repositories to reported table does not exist.
Sampling supplement	Both arms sample, the measured arm over repositories and releases and the anticipatory arm over generated code and schemes.	No sampling rationale is written for either arm, and the prevalence weighting that is meant to justify the operation catalogue has no stated procedure behind it.

Not applicable, despite appearances. Experiments with Human Participants does not apply. The spurious gain experiment uses assistants to perform edits on modules, and no human subject is studied, so a reviewer who expects participant recruitment, consent and demographic reporting would be applying the wrong checklist. Where two raters code documentation or configurations, they are instruments rather than subjects, and agreement is reported as instrument reliability rather than as a finding about people. Case Study does not apply, since no single organisational setting is studied in depth and the claims are population-level rather than contextual. Qualitative Surveys and Grounded Theory do not apply, because the coding schema for operation classes is fixed in advance from configurations rather than developed from the data. Questionnaire Surveys does not apply, since no instrument is administered to anyone.

How It Will Be Evaluated

Metric	Measures	Why this construct	Source	What counts as good
Per-carrier retention under a single operation class	Whether an origin claim present before an operation is still recoverable after it, per carrier and per operation class.	The atomic result. Every other number in the paper composes from it, and it is the only place where carrier and operation are separated cleanly enough to attribute a loss to a cause.	New, defined in this paper. Recovery is defined per carrier using that carrier's own verification procedure, being trailer presence for trailers, signature verification for signatures, attestation resolution for attestations, and the scheme's own detector at the scheme's own threshold for watermarks.	Descriptive. Both high and low retention are results. The value is in which operation classes cluster where, and in whether the classes that destroy most carriers are also the common ones.
Composed retention under observed operation orders	Whether an origin claim survives a sequence of operations applied in an order observed in real repositories.	Single-operation results may not compose, and no release passes through only one operation. This is the number that describes what actually reaches a consumer.	New, with orders derived from repository and workflow configurations rather than constructed by us.	Descriptive. A composed result substantially worse than the product of the single-operation results would be the more notable finding, since it would mean operations interact rather than merely accumulate.
Spurious gain rate	The share of modules that are substantially human under the fixed rule whose release nevertheless carries an origin claim attributing the work to a model.	The second failure direction, absent from the adversarial literature because an attacker has no motive to add a claim, and the direction with the more serious consequence for attribution and for disclosure.	New, defined in this paper. The substantially-human rule is fixed in advance and reported with a sensitivity analysis.	Descriptive, and any non-trivial rate is a headline. The rate must be reported against the threshold used, since a single number without the threshold is uninterpretable.
Operation prevalence	How commonly each operation class appears across sampled repositories and releases.	It weights the retention results by how much they matter in practice, and it is what makes the operation catalogue defensible as observed rather than chosen.	New, from configuration mining, with the sampling frame reported alongside.	Descriptive. The check that matters is whether the catalogue covers the operations that dominate practice, so a long tail of rare operations is a weaker result than full coverage of the common ones.

Metric	Measures	Why this construct	Source	What counts as good
Recovery variance across repeated runs	The spread in recovery outcomes across repeated measurement at identical settings, reported separately for deterministic carriers and statistical ones.	Watermark detection is statistical and generation is stochastic, so unreported variance in the anticipatory arm would undermine every number drawn from it. For deterministic carriers, non-zero variance is itself a defect in the measurement setup and must be investigated rather than averaged away.	Standard practice for stochastic measurement, applied per arm.	Reported rather than minimised in the anticipatory arm, where high variance is a finding about the reliability of provenance claims. Expected to be zero in the measured arm, and any departure is treated as an instrument fault.
Implied misclassification for trailer-based mining	The error that the measured loss and gain rates imply for studies that classify commits or pull requests by trailer presence.	It carries the result out of this paper and into the literature that depends on the signal, which is what makes the study matter to a software engineering audience rather than to a provenance audience only.	Derived, by applying the measured rates to the classification procedure used in published mining studies.	Descriptive, and reported as a range rather than a point, since it depends on the workflow mix of the repositories a given study sampled. A demonstration that the implied error is small would also be a result and would settle the question in the other direction.

State of the Art and References

Checked September 2026, and three findings change what this paper can claim relative to the retired issue. First, the base paper is smaller than it looked. Suresh and colleagues appeared as a two-page contribution in the Second Tiny Papers Track at ICLR 2024, later expanded on arXiv, and the earlier brief cited only the arXiv entry and treated it as a full study. It is also machine learning work rather than software engineering work, so the venue and the audience differ from ours, but the students should not lean on that difference as though it were the whole novelty argument. Second, the trailer layer is live and already mined. A validated multi-method census across 180 million repositories separates agent-authored from agent-trailer commits and reports per-cell precision with confidence intervals and a relative-recall analysis of undercount, so the question of whether the trailer signal is trustworthy as emitted is partly answered and is not available to us as a contribution. Studies of agent pull requests report wide variation in trailer use across agents, one using it in 1.2 percent of its pull requests while others rely on it in over 89 and 95 percent, and a September 2026 longitudinal study reports 62 percent of substantive edits carrying a trailer with a small audit finding 4 percent false positives and 8 percent false negatives. That same study states that it cannot disentangle actual involvement from trailer-generation practice and squash workflows, which is the gap this paper takes, named by someone else and very recently. Third, the scheme space is crowded and a new scheme is not available to us. STONE, SrcMarker, ACW, CodeMark, CodeIP, CLASP and PromptMark occupy it, and a systematic review of watermarking and attribution for AI-generated code, published in an Elsevier journal in 2026, concludes that current techniques suit clean environments and that the field has favoured accuracy and invisibility over long-term resilience. On the packaging layer, in-toto and SLSA are specified and platform attestation support has been moving towards default for public repositories, but the claim about default behaviour comes from a practitioner source and must be confirmed against platform documentation before it appears in the paper. What remains for the students, with the first as the collision that would make the study redundant. Any 2026 work measuring what happens to commit trailers, signatures or attestations across merge, rebase or rebuild, in any venue, since this field moves in months and the gap was named this month. Then the forward citations of the census paper, since anyone building on it is the nearest competitor. Then the supply chain and reproducible build literature, for whether attestation breakage under rebuild has already been characterised in the security community under a different name. Then the disclosure and provenance regulatory sources, since the motivation should cite the obligation and its conditional application to source code rather than assert that provenance matters.

Technical Tasks

Task	What to produce	Done when	Cost
Build the configuration miner that derives the operation catalogue from repository and workflow configurations, recording per repository which operation classes are configured and in what order they run.	Catalogue of operation classes with per-repository evidence, and a prevalence ranking.	The catalogue regenerates from the repository list without manual editing, and every operation class in it traces to a configuration file that is cited.	Medium
Build the per-carrier recovery checks, being trailer extraction, signature verification, and attestation and bill-of-materials resolution, each returning retained, lost or gained rather than a single boolean.	Working recovery component with a test suite over hand-constructed cases.	Each check returns the correct outcome on a hand-built fixture set covering every carrier and every outcome, including the gained case.	Medium
Establish the before state for each carrier, so that a claim recorded as lost is shown to have been present rather than never emitted.	Procedure and implementation that reconstructs the pre-operation state from history.	The procedure distinguishes never-emitted from emitted-then-destroyed on a labelled sample, and the error rate of the distinction is reported.	High
Assemble the repository and release sample from the agreed frame, and record the exclusion reasons for everything dropped.	Repository list, release list, and an exclusion log.	The sample is reproducible from the stated frame and the exclusion log accounts for every candidate considered.	Medium
Run the single-operation measurement across the measured arm.	Retention table over deployed carriers and operation classes.	Every cell is populated or explicitly marked inapplicable with a reason, and the run regenerates from the scripts.	Medium
Derive observed operation orders from the configurations and run the composed measurement.	Composed retention results with the order that produced each one.	Each composed result names the configurations its order was derived from.	Medium
Pin scheme, tool, model and generation settings for the anticipatory arm and record them as part of the artifact.	Version and settings manifest.	A rerun from the manifest reproduces the pilot numbers within reported variance.	Low
Run a pilot of the anticipatory arm on a small subset and report variance.	Pilot results with variance across repeated runs.	The number of repeated runs required for stable detection is determined from the pilot and recorded.	Low
Run the anticipatory arm across the selected schemes and operation classes.	Retention table over undeployed carriers.	Populated with variance reported per cell.	High
Run the spurious gain experiment, using the fixed substantially-human rule, and the sensitivity analysis across plausible thresholds.	Gain rate with threshold sensitivity.	The rate is reported against every threshold in the agreed sensitivity range, not only the chosen one.	Medium

Task	What to produce	Done when	Cost
Compute the implied misclassification for the trailer-based classification procedures used in published mining studies.	Implied error range with the assumptions behind each end of it.	The computation names which published procedure each figure applies to.	Low
Package the artifact, being catalogue, setup, sample, manifest and results, for anonymous deposit.	Deposited artifact with README following the lab checklist.	A reader can regenerate the retention table from the deposited material alone.	Medium

Research Tasks

1. MUST. Run the collision search before anything is built, specifically for 2026 work measuring what merge, rebase, squash or rebuild do to commit trailers, signatures or attestations, in software engineering and in security venues. The gap was named this month, which means someone else may already be taking it. Every technical task waits on this.
2. MUST. Define the three outcomes, being retained, silently lost and spuriously gained, per carrier, in operational terms that a script can apply. The recovery checks and the before-state procedure cannot be built until this exists, and a definition written after the first results would not be trusted.
3. MUST. Fix the rule for a substantially human module, with the threshold and the sensitivity range stated in advance. The spurious gain experiment cannot be designed before this, and fixing it afterwards would invalidate the headline number.
4. MUST. Define the sampling frame for repositories and releases, covering the population, the strategy and the exclusion criteria. The Repository Mining and Sampling standards both fail without it, and the sample assembly task waits on it.
5. Write the positioning against the validated census explicitly, stating that the question of whether the trailer signal is trustworthy as emitted is theirs and that the question of what happens to it afterwards is ours. A reviewer who knows that paper will look for this in the first two pages.
6. Argue the scheme selection for the anticipatory arm, covering availability, reproducibility and whether each represents a distinct embedding strategy, and record why each rejected scheme was rejected.
7. Attempt the in-toto composition frame once, and test whether it yields a prediction about which operation classes destroy which carriers. If it does, restructure the analysis around testing the prediction. If not, state the study as empirical and stop, rather than carrying a frame that does no work.
8. Write the regulatory motivation as a conditional, since whether source code falls within the marking obligation is contested. Assert the consequence if it does, and state the position plainly if it does not.
9. State early and in the abstract that the paper proposes no scheme, since a reviewer who expects one will read the study as incomplete.
10. Run the exhaustive literature pass and produce the sheet, with the strength we can use and the weakness we can address as columns, covering code watermarking, trailer-based mining, supply chain provenance and reproducible builds.
11. Draft threats to validity early, covering the sampling frame, the before-state reconstruction, scheme selection, threshold sensitivity, generation settings, and the substantially-human definition.
12. Write the implications section for empirical software engineering rather than for provenance alone, since that is what carries the paper to an FSE audience.

Ordering and Dependencies

The four MUST research tasks block most of the build. The collision search blocks everything, since a positive result changes the paper rather than the schedule. The three-outcome definitions block the recovery checks and the before-state procedure, because a script cannot record an outcome that has not been defined. The substantially-human rule blocks the spurious gain experiment entirely. The sampling frame blocks sample assembly, and through it every measurement in the measured arm. Running the other way, the anticipatory pilot blocks the full anticipatory run, since it determines how many repeated runs stable detection requires, and running the full matrix first would waste the compute. The before-state procedure is the task that invalidates earlier runs if it is done late, because every retention number computed before it exists rests on an unchecked assumption that the claim was present to begin with, so it belongs early rather than at the end. The positioning against the census should be written before the results are in, since discovering the overlap while writing the discussion is the expensive order.

Decisions to make

1. Propose the split between the measured and anticipatory arms in the paper, being how much space each gets and whether the anticipatory arm appears as a full arm or as a bounded section. Give the recommendation, the alternatives, and what each costs in reviewer risk and in work.
2. Propose which carriers enter the measured arm. Trailers are certain. Signatures and attestations each add a layer with its own verification machinery, and dropping one is a legitimate scope cut. State what each adds to the compositionality claim and what dropping it costs.
3. Propose the sampling frame, with the population, the strategy and the exclusions, and with the alternatives you rejected and why. This one changes what the paper can claim about practice, so it is not a purely technical choice.
4. Propose the substantially-human threshold and its sensitivity range, with the argument for the chosen value and what a reviewer would say about each alternative.
5. Propose what to do if the collision search finds the destruction question already taken. State the fallback, which may be the spurious gain direction alone, and what it costs as a standalone paper.
6. Propose the venue fallback if the work does not reach submission shape, covering whether it goes as a shorter paper or to a later round, and what each route means for the artifact.

Revision history

Issue	What changed
13 September 2026	Reissued on a new argument. The unit moves from a code snippet to a shipped release, and the study now covers every provenance carrier a release holds rather than code watermarks alone. A measured arm on commit trailers, signatures and attestations replaces a study that rested entirely on schemes nobody deploys. Code watermarking is demoted to the anticipatory arm. The base paper relationship is restated, since Suresh et al. is a two-page Tiny Paper rather than the full study the earlier issue implied. The title keeps the question form of the first issue, with the mark pluralised so that it carries the compositionality claim, and with a subtitle that names both failure directions rather than loss alone. The mark here means any origin claim a release carries, and the pipeline means release assembly rather than the build toolchain.
September 2026	First issue, titled Does the Mark Survive the Pipeline. Retired rather than replaced. Its framing measured code watermark persistence under non-adversarial pipeline stages, which sits too close to a published adversarial result and rests on a carrier no tool emits.

References

1. Suresh, T., Ugare, S., Singh, G., and Misailovic, S. Is Watermarking LLM-Generated Code Robust? Second Tiny Papers Track at ICLR 2024. Expanded version arXiv:2403.17983.
2. Marking Code Without Breaking It: Code Watermarking for Detecting LLM-Generated Code (STONE). Findings of EACL 2026.
3. Yang, B., et al. SrcMarker: Dual-Channel Source Code Watermarking via Scalable Code Transformations. IEEE Symposium on Security and Privacy, 2024.
4. Kirchenbauer, J., Geiping, J., Wen, Y., Katz, J., Miers, I., and Goldstein, T. A Watermark for Large Language Models. ICML 2023.
5. CLASP: Training-Free LLM-Assisted Source Code Watermarking via Semantic-Preserving Transformations. arXiv:2510.11251.
6. PromptMark: A Prompt-Guided Iterative-Feedback Framework for Source Code Watermarking. arXiv:2606.20835.
7. Can We Trust the Source? A Systematic Review of Watermarking and Attribution for AI-Generated Code. Information and Software Technology, 2026.
8. Detecting AI Coding Agents in Open Source: A Validated Multi-Method Census of 180 Million Repositories. arXiv:2606.24429.
9. Beyond Simpson's Paradox: A Cascade of Confounders in AI Agent Pull-Request Co-Authorship. arXiv:2606.22711.
10. Who Maintains Agent Skills? A Longitudinal Study of Human-Governed, AI-Assisted Skill Maintenance. arXiv:2609.05677.
11. Torres-Arias, S., Afzali, H., Kuppusamy, T. K., Curtmola, R., and Cappos, J. in-toto: Providing Farm-to-Table Guarantees for Bits and Bytes. USENIX Security Symposium, 2019.
12. Supply-chain Levels for Software Artifacts (SLSA) Specification. <https://slsa.dev/>
13. Ralph, P., Hoda, R., and Treude, C. ACM SIGSOFT Empirical Standards. ACM, 2020. <https://www2.sigsoft.org/EmpiricalStandards/>

AI Disclosure

The original ideas, analysis, and research synthesis in this report are the intellectual contribution of the authors. AI-assisted tools were used to enhance the clarity and presentation of the writing.